COMPLETE GUIDE · FOLDER STRUCTURE

SKY KART 3D
ISLAND GRAND PRIX

Every folder and every file of the game, what each one does, how they work together, and how to build, run, test and host it.



12	5	20	34	0
tracks	karts	missions	source files	downloaded art

Written with Claude, 23 October 2026, from the project folder D:\Game\Sky Kart 3D. The game was tested by programs and in headless Chrome (desktop and phone emulation). It has not been tried on a real phone yet (see chapters 7 and 8).

Contents

1. The game at a glance 4
- The game in one table 4
- What was built, step by step 5
- What is finished, and what is not tested yet 5
- How to read this guide 5

2. The full folder structure 6
- The whole tree 6
- The folders in one table 7
- Top-level files 8
- src/ - the start, the state and the race screen 8
- src/ui/ - the menu screens 9
- src/game/ - data, rules, simulation, 3D, sound, input 9
- public/ - copied into the finished game unchanged 11
- scripts/ - tests and tools 11
- docs/ - the documents 12
- dist/ - the finished game 12
- node_modules/ - the libraries 12

3. How the files work together 14
- The five layers 14
- The screens and how you move between them 15
- One frame of a race 16
- From track data to a 3D road 17
- How the game knows where a kart is 17
- Laps and checkpoints 17
- The save 17
- Rules every part agrees on 18

4. Building the game step by step 19
- What you need 19
- Creating the empty project 19
- The build order at a glance 19
- Phase 0: the skeleton 20
- Phase 1: the data 20
- Phase 2: the track builder 20
- Phase 3: the simulation 21
- Phase 4: the 3D world 21
- Phase 5: HUD and touch 22
- Phase 6: rules and saving 22
- Phase 7: the screens 22
- Phase 8: sound 22
- Phase 9: twelve tracks 22
- Phase 10: phone and offline 23
- Phase 11: tests, tuning and documents 23

5. What the game can do 24
- The main menu and the map 24
- The twelve tracks 25
- A race 27

Stars, rewards and unlocking 28
Missions 28
The garage 29
Controls 30
Pause and settings 31
Sound and music 32
Saving 32

6. Running, building and hosting 33
Play on your Windows PC 33
Play on your phone 33
All the commands 33
Putting the game online 34
When something goes wrong 34

7. Testing 36
The test programs and their results 36
What each program checks 36
Difficulty 38
Speed 38
What has NOT been tested 38
How to run all the tests again 39

8. Changing and extending the game 40
Where to change what 40
How to add a new track 40
How to add a new kart 41
How to add a new mission 41
How to add a new screen 41
Known limits 41
Ideas for later 41

9. Words used in this guide 43

1. The game at a glance

Sky Kart 3D: Island Grand Prix is a 3D kart racing game that runs in a web browser, on a PC or on a phone. You race three computer drivers around 12 tracks built on floating islands in the sky. You collect coins, drift for boosts, jump off ramps, find a shortcut on every track, finish missions, and spend your coins in a garage on new karts, upgrades and paint.



A race on Tropical Coast. The minimap is top right, the boost bar is at the bottom.

This guide explains **how the game is built**: every folder and every file, what each one does, how they work together, the order to build them in, how to run and host the game, and how it was tested.

The game in one table

What	Value
Name	Sky Kart 3D: Island Grand Prix
Kind of game	Arcade kart racer, single player against 3 computer drivers
Where it runs	Any modern browser with WebGL: PC (keyboard or gamepad) and phone or tablet (touch)
Built with	Vite 5, React 18, JavaScript, React Three Fiber 8, Three.js 0.169, Zustand 4, Howler 2
Tracks	12, each with its own shape, theme, weather, scenery, surfaces, obstacles and a shortcut
Race	4 karts, 3 laps, countdown, ordered checkpoints, live positions
Garage	5 karts, 4 upgrades with 5 levels each, 8 paints, 5 drivers, 4 wings
Progress	Stars (1 to 3 per track), 3 tasks per track, 20 missions, coins, best times
Art and sound	All made in code. No downloaded images, models, fonts, sounds or music
Size of the finished game	1.6 MB in `dist/` (about 330 KB of zipped JavaScript and CSS)

What	Value
Saving	In the browser (`localStorage`), nothing is sent anywhere
Project folder	`D:\Game\Sky Kart 3D`

What was built, step by step

1. **The rules first, without graphics.** Track data, the track builder and the race simulation were written as plain JavaScript that also runs in Node.js. This way every track could be checked by a program before anything was drawn.
2. **Twelve tracks as data.** Each track is a list of points of a smooth closed curve plus a theme. A placement module finds good spots for boost pads, ramps, sand, ice, swinging boulders, coins and the shortcut.
3. **The 3D world.** Road, walls, floating island, scenery and karts are built from simple shapes with colours painted on the corners (no pictures), so the game stays small and fast.
4. **The screens.** Main menu, world map, garage, missions, settings, pause and results, in React.
5. **Saving and rewards.** One save object, checked and repaired when it is loaded, written to the browser after every reward or purchase.
6. **Touch controls, sound and music**, all made in code.
7. **Tests.** Programs check the tracks, the rules, the save, full AI races, the gameplay and a real browser.

What is finished, and what is not tested yet

Everything listed in the table above works and is checked by the test programs (see chapter 7).

> **Honest status:** The game has **not** been tried on a real phone yet. Touch was tested only by a phone emulator inside Chrome. The frame rate on real devices has not been measured, the sound has not been listened to by a person, and only Google Chrome was used. Chapter 8 lists all known limits.

How to read this guide

If you want to...	Read
See every folder and file and what it does	Chapter 2
Understand how the files talk to each other	Chapter 3
Build the same game yourself, in order	Chapter 4
Know everything the game can do	Chapter 5
Play it, build it or put it online	Chapter 6
See how it was tested	Chapter 7
Change it or add a track, kart or mission	Chapter 8
Look up a word	Chapter 9

2. The full folder structure

This chapter shows **every folder and every file** in the project, in the same order you see them on disk. For each file you get its size in lines, what it does, and which other files use it. The project lives in `D:\Game\Sky Kart 3D`.

The whole tree

The tree leaves out only the files that programs make for you: `node_modules/` (the downloaded libraries), the inside of `dist/` (the finished game, shown in its own section below) and the inside of `scripts/out/` (test results).

```
Sky Kart 3D/
├── README.md                   short introduction and the most used commands
├── package.json                project name, the npm commands, the libraries
├── package-lock.json           exact library versions (written by npm, do not edit)
├── vite.config.js              build settings: relative paths, 3 extra chunks
├── index.html                  the only HTML page; it loads src/main.jsx
├── play now.cmd                Windows launcher: install, build, start, open
│
├── src/                        THE GAME (source code)
│   ├── main.jsx                starts React, registers the offline service worker
│   ├── App.jsx                 shows the right screen, starts the menu music
│   ├── store.js                app state: screen, save, pause (Zustand)
│   ├── styles.css              the look of every screen, the HUD and the touch buttons
│   ├── Race.jsx                the race: 3D canvas, game loop, pause, focus loss
│   ├── Hud.jsx                 position, lap, timer, coins, minimap, boost bar
│   ├── Touch.jsx               joystick + DRIFT / BOOST / BRAKE (/ GAS) buttons
│   │
│   ├── ui/                     the menu screens (React)
│   │   ├── Shell.jsx           shared frame: logo, coins, bottom bar, stars, sky
│   │   ├── Menu.jsx            main menu with the 3D kart
│   │   ├── MapScreen.jsx       world map: 12 track cards + the detail panel
│   │   ├── Garage.jsx          karts, drivers, wings, upgrades, paint
│   │   ├── GaragePreview.jsx   the turning 3D kart on its stand
│   │   ├── Missions.jsx        20 missions with progress bars and CLAIM
│   │   ├── SettingsScreen.jsx  the settings page
│   │   ├── Pause.jsx           pause menu + the settings panel (sliders, toggles)
│   │   └── Results.jsx         finishing order, time, rewards, next buttons
│   │
│   └── game/                   everything that is not a screen (no React here)
│       ├── tracks.js           DATA: the 12 tracks
│       ├── karts.js            DATA: karts, paints, drivers, wings, upgrades, rivals
│       ├── progress.js         RULES: rewards, stars, unlocking, missions, shop
│       ├── save.js             RULES: save format, repair, load, write
│       ├── autoplace.js        TRACK: finds where pads, ramps, hazards, shortcut go
│       ├── trackBuild.js       TRACK: curve -> road, checkpoints, "where is the kart"
│       ├── setup.js            RACE: makes the 4 racers of a race
│       ├── sim.js              RACE: the simulation, 60 steps a second
│       ├── ai.js               RACE: the computer drivers
│       ├── input.js            keyboard + gamepad + touch -> one input object
│       ├── audio.js            all sounds and music, made in code
│       ├── bus.js              small shared object for the HUD and the tests
│       ├── geo.js              3D: helpers for simple coloured shapes
│       ├── kartModel.js        3D: the kart and its driver
│       ├── props.js            3D: scenery (palms, pines, mushrooms, towers...)
│       ├── biomes.js           3D: which scenery a track uses
│       ├── world3d.js          3D: road, walls, island, scenery (built once)
│       └── visuals.js          3D: karts, coins, camera, sky, weather, sparks
│
├── public/                     copied unchanged into the finished game
│   ├── icon.svg                the app icon
│   ├── manifest.webmanifest    name and colours for "Add to home screen"
│   ├── sw.js                   service worker: lets the game start offline
│   └── previews/               12 track pictures for the map (tropical.jpg ...)
```

```
│
├── scripts/                    tools and tests (Node.js); not part of the game
│   ├── lib/
│   │   ├── cdp.mjs             starts and drives a hidden Chrome
│   │   └── network.mjs         finds the PC's Wi-Fi address and a free port
│   ├── check-tracks.mjs        TEST: shape of all 12 tracks
│   ├── check-save.mjs          TEST: save repair and every rule
│   ├── check-sim.mjs           TEST: full AI races on all 12 tracks
│   ├── check-gameplay.mjs      TEST: laps, ramps, boost, stats, shortcuts
│   ├── browser-test.mjs        TEST: the real game in Chrome, desktop and phone
│   ├── bot-balance.mjs         difficulty check with simulated players
│   ├── perf-report.mjs         draw calls, triangles, JS time on every track
│   ├── play.mjs                serves the game to this PC and to a phone
│   ├── make-previews.mjs       remakes public/previews
│   ├── tour.mjs                a picture of every track
│   ├── shot.mjs                quick pictures of a few screens
│   ├── screens.mjs             a picture of every menu screen
│   ├── guide-shots.mjs         the pictures in this guide
│   ├── build-guide.mjs         makes this PDF
│   └── out/                    results and pictures written by the scripts
│
├── docs/                       documents
│   ├── 01-controls-and-features.md
│   ├── 02-architecture.md
│   ├── 03-setup-and-deploy.md
│   ├── 04-testing-report.md
│   ├── 05-asset-credits.md
│   ├── 06-known-issues.md
│   ├── guide/                  the chapters of this PDF
│   │   ├── 01-overview.md ... 09-words.md
│   │   ├── images/             pictures taken from the game
│   │   └── diagrams/           the drawings (SVG)
│   └── Sky-Kart-3D-Complete-Guide.pdf     this document
│
├── dist/                       THE FINISHED GAME (made by "npm run build")
└── node_modules/               libraries (made by "npm install", about 75 MB)
```

The folders in one table

Folder	What is in it	Files	Lines	Who writes it
(top level)	settings, launcher, the HTML page	7		you
`src/`	the game screens that are not menus, the state, the styles	7	675	you
`src/ui/`	the menu screens	9	531	you
`src/game/`	rules, data, simulation, 3D, sound, input	18	2 916	you
`public/`	icon, offline worker, track pictures	3 + 12 pictures	55	you (pictures: a script)
`scripts/`	tests and tools	15 + 2 in `lib/`	about 1 500	you
`docs/`	documents and this guide	6 + guide		you
`dist/`	the finished game to put online	12 + 12 pictures		`npm run build`
`node_modules/`	React, three.js and the other libraries	thousands		`npm install`
`scripts/out/`	test logs and screenshots	varies		the scripts

All the game's own code is **34 files and about 4 100 lines** in `src/`. The lines are long and dense, so this is about 243 KB of text.

> **Rule:** Only `src/`, `public/`, `scripts/`, `docs/` and the top-level files are written by hand. Never edit `dist/` or `node_modules/`: they are thrown away and made again by `npm run build` and `npm install`.

Top-level files

File	Lines	What it does
`README.md`	44	First page of the project: what the game is, how to play it, the list of documents, the test commands.
`package.json`	30	The project's name and version, the **npm commands** (`dev`, `build`, `preview`, `play`, `check`, `test:browser`, `previews`, `balance`, `guide`) and the libraries with their versions.
`package-lock.json`	2 042	Written by npm. It pins the exact version of every library so the game builds the same way on every PC. Do not edit it.
`vite.config.js`	17	Settings for Vite, the build tool. `base: './'` makes the finished game work from any web folder. It also splits three.js, React Three Fiber and Howler into their own files so the menus load first and fast.
`index.html`	18	The single web page. It sets the phone viewport (no zoom, fills the notch area), links the icon and the manifest, and loads `src/main.jsx`.
`play now.cmd`	83	Double-click launcher for Windows. It checks Node.js, runs `npm install` the first time, builds the game, then starts `scripts/play.mjs`, which opens the browser and prints a phone address with a QR code.

src/ - the start, the state and the race screen

File	Lines	What it does	Used by
`main.jsx`	13	The starting point. Creates the React root inside `index.html`, puts the store on `window.__store` (for tests) and registers the service worker `sw.js`.	`index.html`
`App.jsx`	48	Shows exactly one screen, chosen by `store.screen`: menu, map, garage, missions, settings, race or results. Starts sound after the first tap or key press (browsers require that) and plays menu music outside races. Loads the 3D race code only when it is needed.	`main.jsx`
`store.js`	70	The **app state**, made with Zustand: which screen is open, the save, the selected track, pause, the last results. Every action that changes the save (finish a race, claim, buy, equip) goes through here and is written to the browser at once. Also decides the "auto" graphics quality for this device.	every screen
`styles.css`	190	All the styling: sky background, buttons, cards, the garage, the HUD, the touch buttons, phone sizes (landscape and portrait), notch safe areas, reduced-motion support.	all screens
`Race.jsx`	196	The race screen. Makes the 3D canvas, builds the race world, and runs the **game loop** every frame: read input, run the simulation, update the 3D scene, play sounds. Pauses on Esc, on focus loss and when the tab is hidden. Lowers the picture resolution if the frame rate drops.	`App.jsx`
`Hud.jsx`	91	The race display drawn on top of the 3D picture: position (1st/4), lap, timer, coins, minimap, boost bar, "WRONG WAY", short messages ("LAP 2 / 3"), the pause button and the key hint. It reads numbers 10 times a second; it does not slow the race down.	`Race.jsx`

File	Lines	What it does	Used by
`Touch.jsx`	67	Touch controls: a joystick on the left (the further you move it, the more the kart turns) and big DRIFT, BOOST and BRAKE buttons on the right (GAS too if auto-accelerate is off). Several fingers work at the same time.	`Race.jsx`

src/ui/ - the menu screens

File	Lines	What it does
`Shell.jsx`	55	Parts shared by the menu screens: the logo, the coin counter, the stars, the floating islands in the sky, and the frame with the bottom bar (Campaign, Garage, Missions with a red count of claimable rewards, Settings).
`Menu.jsx`	40	The main menu: logo, **Start / Continue Adventure** (shows the next track), World Map, Garage, Missions, Settings, the coins and stars, and your kart turning on its stand.
`MapScreen.jsx`	68	The world map: 12 track cards with a picture, number, stars and a lock. The panel on the side shows the chosen track: picture, laps, shortcut name, description, the 3 tasks, best time and best lap, and **START RACE** (only if the track is open).
`Garage.jsx`	138	The garage with four tabs. **Kart**: browse the 5 karts, see their stats, buy or equip, and choose paint. **Driver** and **Wings**: buy and equip. **Upgrades**: 4 upgrades with 5 levels each, the price of the next level and the new stats.
`GaragePreview.jsx`	41	A small 3D scene with the kart on a turntable. It turns by itself; you can drag it to turn it. Used by the garage and the main menu.
`Missions.jsx`	39	The list of 20 missions with a progress bar and the reward. Finished missions get a **CLAIM** button; claimed ones show a tick.
`SettingsScreen.jsx`	21	The settings page. It shows the settings panel from `Pause.jsx` inside the menu frame, plus **Reset all progress** (it asks "Delete ALL progress, coins and karts?" before doing it).
`Pause.jsx`	72	The **pause menu** (Resume, Restart race, Settings, Quit to map) and the **settings panel** used in two places: master / music / effects volume, mute, auto-accelerate, steering sensitivity, vibration, show FPS and graphics quality.
`Results.jsx`	57	After a race: big position, stars, the 4 finishers with times, your time ("NEW BEST TIME!"), each reward line and the total, the 3 track tasks, "New track unlocked", and **NEXT TRACK**, **REPLAY**, **MAP**.

src/game/ - data, rules, simulation, 3D, sound, input

This folder holds the game itself. None of these files use React. Many of them (marked **Node**) do not use the browser at all, so the test programs can run them directly.

File	Lines	Node	What it does
`tracks.js`	239	yes	The **12 track definitions** as plain data: name, description, difficulty, the 3 tasks, road width, the curve points, colours (sky, fog, ground, road, walls), weather, scenery list, landmarks, music, and a recipe for pads, ramps, hazards and the shortcut.
`karts.js`	100	yes	The **shop catalogue**: 5 karts with prices, weights and stats; 8 paints; 5 drivers; 4 wings; upgrade prices; and `simStats()`, which turns stats (1 to 10) into real numbers: top speed, acceleration, steering speed, boost. Also the 6 rival drivers and how strong they are on each track.
`progress.js`	140	yes	The **rules that change the save**: coins for each place, stars, unlocking the next track, track tasks, the 20 missions, claiming, buying and equipping. Every function returns a new save and never changes the old one.

File	Lines	Node	What it does
`save.js`	90	yes	The **save**: its shape, the default save, `sanitize()` (turns any broken or edited save into a valid one), loading from and writing to `localStorage`. A broken save is kept aside under a second key.
`autoplace.js`	150	yes	Looks at how curved each part of the road is and **places the gameplay objects**: boost pads and ramps on straights, sand / mud / ice zones, swinging boulders, coin lines, and where the shortcut leaves and rejoins the road. Always the same result for a track (seeded).
`trackBuild.js`	339	yes	Turns a track's curve into **road samples every 2 m** (position, direction, side, curve), builds the shortcut road, checkpoints, walls and coins, and answers "**where is this kart?**" (`locate`): which sample, how far from the centre, on the shortcut or not.
`setup.js`	29	yes	Builds the **4 racers** for a race: you (from the garage: kart, upgrades, paint, driver, wing) and 3 rivals (chosen by the track number).
`sim.js`	417	yes	The **race simulation**, 60 steps per second: countdown, speed, steering, drifting and mini-boosts, the boost bar, pads, ramps and jumps, surfaces, walls, bumps between karts, swinging obstacles, coins, checkpoints, laps, positions, safe recovery, finish and results.
`ai.js`	108	yes	The **computer drivers**: follow the road with a lane of their own, slow down for curves, use boost on straights, sometimes take the shortcut, and stay close to the player (rubber band).
`input.js`	86		Reads the **keyboard, a gamepad and the touch buttons** and merges them into one `input` object (steer, gas, brake, drift, boost, look back, reset). Clears everything when the window loses focus.
`audio.js`	220		**All sound, made in code**: engine, drift, coins, boost, jumps, hits, countdown, menu clicks, and a short music loop per track plus menu music. Uses Howler to play them. Nothing starts before the first tap or key press.
`bus.js`	7		A tiny shared object (`window.__sky`): the race world, short messages, FPS and picture scale. The HUD reads it; the tests use it to drive the game.
`geo.js`	46		Helpers for **low-poly shapes**: boxes, cylinders, cones, spheres and more, each with a colour, merged into one shape. Also a seeded random generator.
`kartModel.js`	92		Builds the **3D kart and driver** from simple shapes, with the chosen paint, helmet colours and wing. Used in the race and the garage.
`props.js`	74		The **scenery models**: palms, pines, cacti, pillars, coral, maple trees, mushrooms, moon rocks, towers and more, plus big landmarks (lighthouse, pyramid, volcano, castle...).
`biomes.js`	2		Says which scenery models a track uses (keeps `world3d.js` general).
`world3d.js`	477		Builds the **world that does not move**, once per race: road, kerbs, walls, start gate, checkpoint flags, pads, ramps, the shortcut road, the floating island with its cliffs, scenery (thousands of copies drawn in a few batches), landmarks, small islands around.
`visuals.js`	300		Everything that **moves on screen**: karts (smoothly between simulation steps), coins, swinging boulders, the chase camera, the sky and sun, clouds, weather (snow, rain, embers, leaves...), lightning, aurora, sparks and dust.

public/ - copied into the finished game unchanged

File	What it does
`icon.svg`	The game icon (browser tab and home screen).
`manifest.webmanifest`	Name, colours and icon for "Add to home screen" on phones. Landscape is preferred.
`sw.js`	The **service worker**: after the first visit it keeps a copy of the game so it can start without internet. New versions are picked up when online.
`previews/*.jpg`	12 pictures of the tracks seen from above, shown on the map. Made by `scripts/make-previews.mjs` from the game itself.

scripts/ - tests and tools

These are run with Node.js from the project folder. They are not part of the game and are not uploaded.

File	Lines	What it does	Needs
`lib/cdp.mjs`	76	Starts a hidden Chrome, opens the built game and lets a script press keys, touch the screen, read values and take pictures.	Chrome
`lib/network.mjs`	37	Finds the PC's address on the home Wi-Fi and a free port.	
`check-tracks.mjs`	116	Checks that every track is a closed smooth loop, does not touch itself, has sane corners, a straight start, ordered checkpoints, a shorter shortcut, pads, ramps, coins on the road and tasks.	
`check-save.mjs`	136	27 checks of saving and rules: broken saves, stars, unlocking, no double rewards, missions, purchases, upgrades, equipment.	
`check-sim.mjs`	32	Runs a full 3-lap race on every track with all karts driven by the AI and checks that everyone finishes.	
`check-gameplay.mjs`	167	31 checks of the driving rules: laps need checkpoints, ramps, low gravity, boost, drifting, pads, walls, braking, recovery, bumps, kart stats, and that every shortcut is faster.	
`browser-test.mjs`	255	76 checks in real Chrome: menus, buttons, keys, pause, rewards, saving and reloading, garage, broken saves, settings, all 12 tracks finished, phone touch and screen sizes.	Chrome, `dist/`
`bot-balance.mjs`	48	Races simulated "casual" and "good" players on every track to see how hard the campaign is.	
`perf-report.mjs`	31	Measures draw calls, triangles and JavaScript time on every track at low, medium and high quality.	Chrome, `dist/`
`play.mjs`	56	Serves `dist/` on the PC and the local network, opens the browser, prints the phone address and a QR code.	`dist/`
`make-previews.mjs`	7	Remakes the 12 map pictures in `public/previews` (runs `tour.mjs`).	Chrome, `dist/`
`tour.mjs`	34	Takes a picture of every track, from above or while driving.	Chrome, `dist/`
`shot.mjs`	28	Quick pictures of the menu, map, garage and one race.	Chrome, `dist/`
`screens.mjs`	22	A picture of every menu screen at any screen size (for checking phone layouts).	Chrome, `dist/`

File	Lines	What it does	Needs
`guide-shots.mjs`	68	Takes the pictures used in this guide.	Chrome, `dist/`
`build-guide.mjs`	327	Turns the chapters in `docs/guide/` into this PDF, using Chrome's "print to PDF".	Chrome

docs/ - the documents

File	What is in it
`01-controls-and-features.md`	Keys, gamepad and touch controls; how races, stars, rewards, missions, the garage and saving work.
`02-architecture.md`	A short version of chapters 2 and 3 of this guide.
`03-setup-and-deploy.md`	Install, run, build and put online.
`04-testing-report.md`	What was tested, the results, the performance numbers, what was not tested.
`05-asset-credits.md`	Where every picture and sound comes from (all made in code) and the library licences (all MIT).
`06-known-issues.md`	Known limits and ideas for later.
`guide/`	The 9 chapters of this PDF (Markdown), `images/` and `diagrams/`.
`Sky-Kart-3D-Complete-Guide.pdf`	This document.

dist/ - the finished game

`npm run build` reads `index.html`, everything in `src/` and `public/`, and writes the finished game into `dist/`. This folder is **all you upload** to a web host. The file names inside `assets/` contain a code (for example `index-Ba7Aa88X.js`) that changes whenever the content changes, so browsers never use an old copy.

```
dist/
├── index.html                    1 KB   the page
├── icon.svg, manifest.webmanifest, sw.js      copied from public/
├── previews/                   480 KB   the 12 map pictures
└── assets/
    ├── index-….js              195 KB  (64 KB zipped)   React, menus, rules, track data
    ├── index-….css              23 KB   (6 KB zipped)   all styles
    ├── three-….js              683 KB (175 KB zipped)   three.js (3D engine)
    ├── r3f-….js                138 KB  (45 KB zipped)   React Three Fiber
    ├── Race-….js                74 KB  (30 KB zipped)   race, simulation, 3D world
    ├── audio-….js               36 KB  (10 KB zipped)   Howler + the sound code
    ├── kartModel-….js            8 KB   (3 KB zipped)   the kart model
    └── GaragePreview-….js        2 KB   (1 KB zipped)   the turning kart
```

The whole `dist/` folder is **1.6 MB**, about 330 KB of zipped code. A player first downloads the page, the styles and the menu code (about 70 KB zipped), so the buttons appear quickly. three.js and React Three Fiber follow for the turning kart on the main menu, and the race code is fetched while the player looks at the map, before the first race starts.

node_modules/ - the libraries

`npm install` downloads the libraries listed in `package.json` into this folder (about 75 MB). The game needs six of them; three more are only used while building and testing.

Library	Version	Used for	In the finished game?
react, react-dom	18.3	the screens	yes
three	0.169	3D drawing (WebGL)	yes
@react-three/fiber	8.17	puts the three.js canvas inside React and runs the frame loop	yes
zustand	4.5	the small app-state store	yes
howler	2.2	plays sounds and music on every browser	yes
vite	5.4	development server and build	no
@vitejs/plugin-react	4.3	lets Vite understand `.jsx`	no
qrcode-terminal	0.12	prints the phone QR code in `play now.cmd`	no

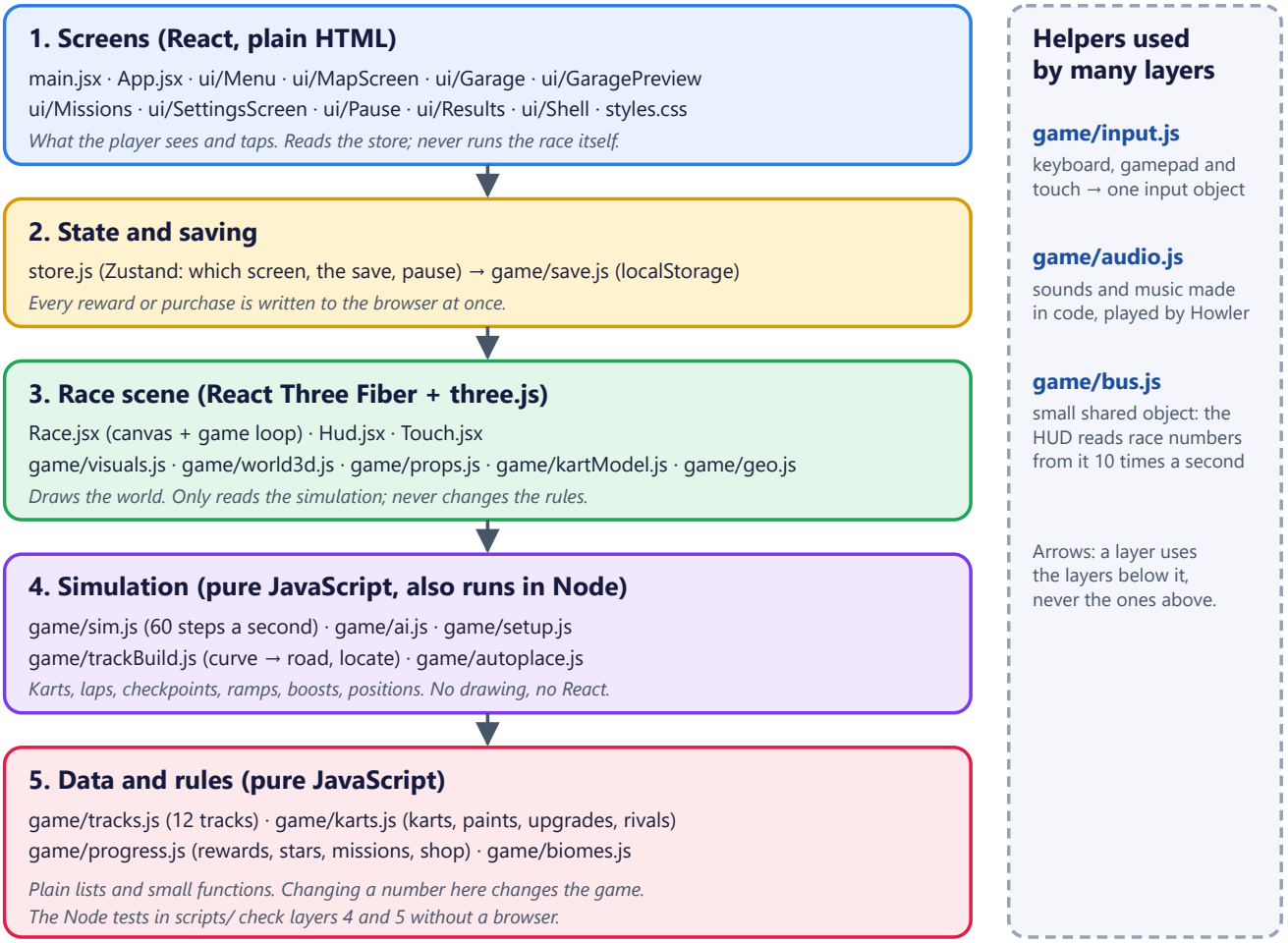
All of them use the MIT licence.

3. How the files work together

Chapter 2 listed the files one by one. This chapter shows how they connect: the layers, the screens, what happens in one frame, how a track goes from data to a 3D road, and how the save works.

The five layers

The code is stacked in five layers. A layer may use the layers **below** it, never the ones above. This is the most important rule of the project: it is why the rules and the simulation can be tested in Node.js without a browser, and why the 3D picture can never change the result of a race.

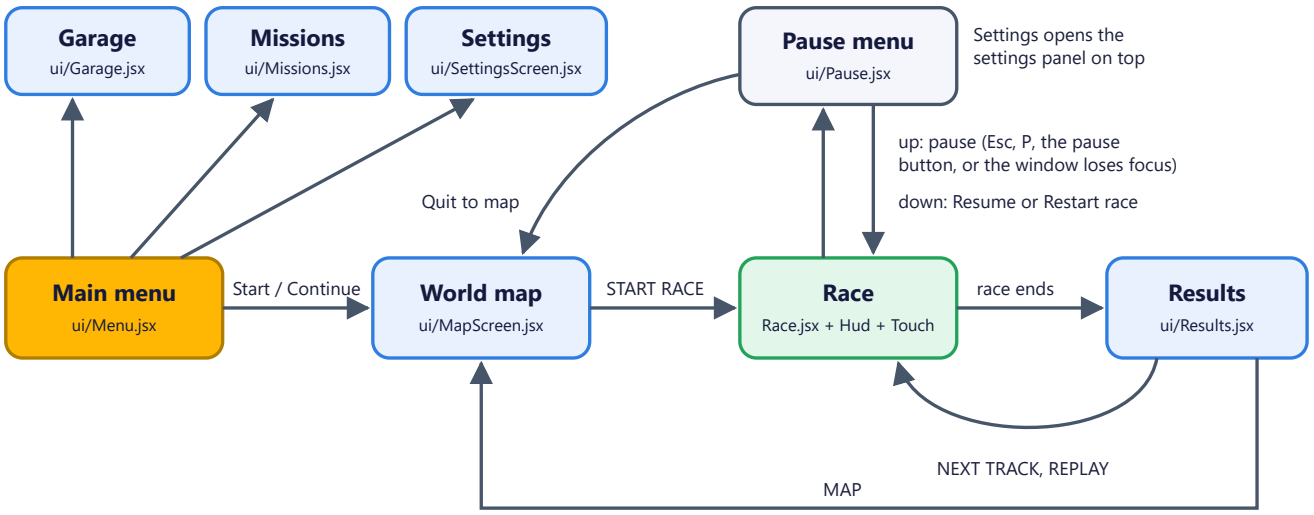


The five layers of the code, top to bottom. The dashed box holds helpers that several layers use.

Layer	Folder	Knows about React?	Knows about three.js?	Runs in Node tests?
1. Screens	`src/ui/`, `App.jsx`	yes	only the garage kart	no
2. State and saving	`store.js`, `game/save.js`	store only	no	save yes
3. Race scene	`Race.jsx`, `Hud.jsx`, `Touch.jsx`, `game/visuals.js`, `world3d.js` …	`Race.jsx` and HUD	yes	no
4. Simulation	`game/sim.js`, `ai.js`, `trackBuild.js`, `autoplace.js`, `setup.js`	no	no	**yes**
5. Data and rules	`game/tracks.js`, `karts.js`, `progress.js`	no	no	**yes**

The screens and how you move between them

`store.js` holds one value, `screen`, and `App.jsx` shows the screen it names. A button only changes that value (for example `go('garage')` or `startRace(3)`).



The map, garage, missions and settings screens also share a bottom bar to jump between them. store.js holds the current screen.

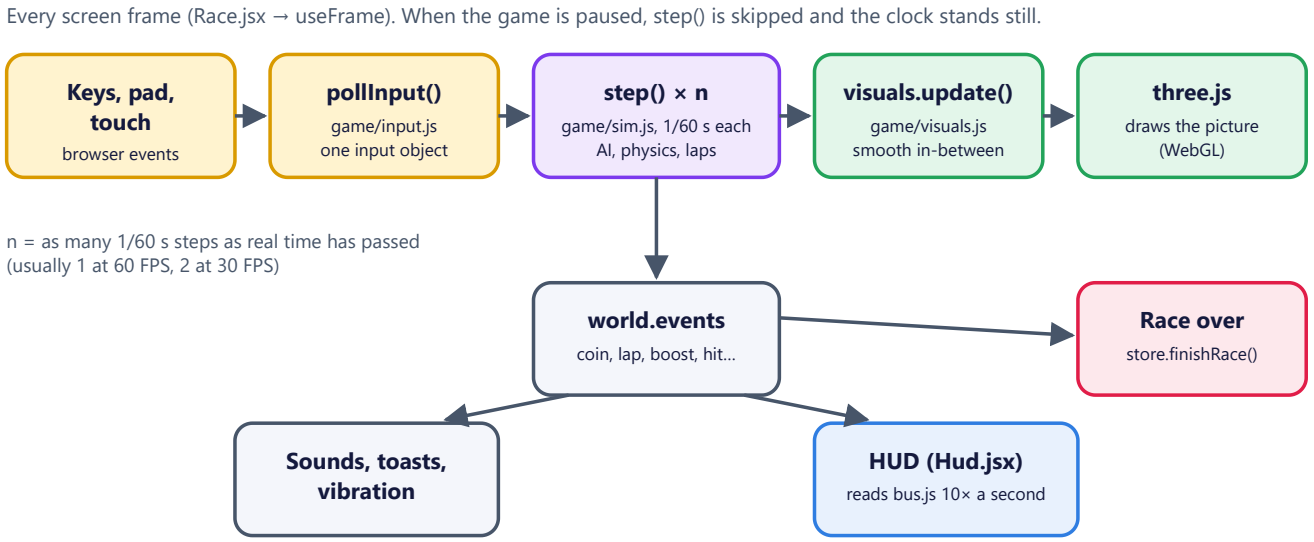
How the player moves between the screens.

Store action	What it does
`go(screen)`	Opens a menu screen (map, garage, missions, settings, menu).
`continueAdventure()`	Opens the map with the first open track you have not finished selected.
`startRace(i)`	Starts track `i`, **only if it is unlocked**. Each start gets a new `raceKey`, so Restart and Replay always begin a fresh race.
`setPaused(true / false)`	Pauses or resumes. `openSettings(true)` opens the settings panel over the race.
`finishRace(world)`	Called once when the race is over: works out the rewards, saves, opens Results.
`nextTrack()`, `replay()`, `quitRace()`	The Results and Pause buttons.

Store action	What it does
`claim`, `buyKart`, `buyPaint`, `buyDriver`, `buyWing`, `buyUpgrade`, `equip...`	Shop and missions. Each one calls a rule in `progress.js` and saves straight away.
`setSetting(...)`, `resetSave()`	Settings.

One frame of a race

While a race is on, `Race.jsx` runs the same short list of jobs every time the screen is redrawn (usually 60 times a second).



What happens in one frame of a race.

1. **Read the controls.** `pollInput()` in `input.js` merges keyboard, gamepad and touch into one object: steer (-1 to 1), gas, brake, drift, boost, look back, reset.
2. **Move the simulation forward.** `sim.step()` always moves time by exactly 1/60 of a second. If the last frame took 1/30 s, it runs twice. This keeps the driving the same on a fast PC and a slow phone. When paused, it does not run at all, so the race clock stops.
3. **Update the picture.** `visuals.update()` places every kart **between** the last two simulation steps, so the movement looks smooth even when the screen rate and the simulation rate do not match. It also moves the camera, coins, boulders, weather and sparks.
4. **React to events.** The simulation leaves a list of things that happened (coin, boost, lap, jump, hit, finish...). `Race.jsx` turns them into sounds, short messages and phone vibration, then empties the list.
5. **Check the frame rate.** Once a second it counts the frames. Below 48 FPS it lowers the picture resolution a little (at most down to 0.55 of a normal screen's pixels per point); above 57 FPS for 5 seconds in a row it raises it again, never above the limit of the chosen quality.
6. **End of race.** When everyone has finished (or 8 seconds after you did), it waits a moment and calls `store.finishRace()`.

The HUD does not take part in the frame. `Hud.jsx` reads a few numbers from the race 10 times a second, so drawing text never slows the 3D down.

From track data to a 3D road

A track starts as about 20 lines of data in `tracks.js` and goes through these steps before you can drive on it.

Step	File	In	Out
1	`tracks.js`	(written by hand)	Curve points, theme colours, weather, scenery, music, tasks, and a recipe: "4 boost pads, 2 ramps, 1 sand zone, a dirt shortcut near the middle of the lap".
2	`trackBuild.js`	the curve points	Road samples every 2 m: position, direction, the sideways direction, how curved the road is.
3	`autoplace.js`	the samples + the recipe	**Where** each object goes. Pads and ramps on straight parts, the shortcut across a real bend where it saves distance and stays away from the rest of the road, zones and boulders spread over the lap, coin lines. A track always gets the same result.
4	`trackBuild.js`	the samples + the places	The full track: pads, ramps, zones, swinging boulders, coins, the shortcut road, 5 or 6 checkpoints, walls.
5a	`sim.js`	the full track	The race uses it for driving, laps and positions.
5b	`world3d.js`	the full track	The 3D road, kerbs, walls, island, scenery and landmarks.

Both the simulation and the 3D world read **the same track object**, so what you see is always exactly where the game thinks it is.

How the game knows where a kart is

`locate()` in `trackBuild.js` answers one question many times a second: for a point (x, z), which road sample is nearest, and how far to the left or right of the centre line is it? From this the game gets:

- **Progress**: how far around the lap the kart is. Positions (1st to 4th) come from laps done plus progress.
- **On or off the road**: off the road the kart is slower (grass); past the wall it is pushed back.
- **The surface** under the kart: road, dirt, sand, water or ice.
- **On the shortcut or not**: the shortcut has its own samples. Its progress is mapped onto the stretch of main road it skips, so laps and positions still use one number.

Laps and checkpoints

Each track has 5 or 6 checkpoints (the blue flags), always outside the shortcut. Each kart remembers the next checkpoint it needs. A lap only counts when the kart crosses the finish line **after all checkpoints, in order**. The position score is also capped at the next checkpoint, so cutting across the grass or driving backwards can never gain places. `check-gameplay.mjs` tests this by moving a kart straight to the end of the lap: the lap is not counted.

The save

The save is one JSON object stored in the browser under the key `skykart3d.save.v1`.

Part	What is in it
`coins`	Coins you can spend.
`unlocked`	How many tracks are open (1 to 12).
`tracks`	For each track you have raced: stars (0 to 3), best time, best lap, the 3 tasks done or not, finished or not.

Part	What is in it
`claimed`	Which missions have been claimed. A mission can never pay twice.
`totals`	Counters for the missions: coins picked up, seconds drifted, pads used, ramps, shortcuts, races, wins, seconds of boost, fastest race.
`garage`	Karts owned, kart in use, paint per kart, paints owned, upgrade levels per kart, driver and wing owned and in use.
`settings`	Volumes, mute, auto-accelerate, steering sensitivity, vibration, show FPS, graphics quality.

Three rules keep the save safe:

1. **Repair on load.** `sanitize()` in `save.js` accepts anything: broken text, missing parts, wrong types, impossible numbers, unknown karts. It always returns a valid save. If the text cannot be read at all, a fresh save is used and the broken text is kept under `skykart3d.save.v1.bad`.
2. **Rules return a new save.** Every rule in `progress.js` (finish a race, claim, buy, equip, upgrade) takes the save and returns a **new** one plus "ok" or a reason. It never changes the old one halfway.
3. **Write at once.** `store.js` writes the new save to the browser in the same moment it accepts it, before the screen changes. If the browser refuses to store it (for example a private window), the game keeps working and shows a warning.

Rules every part agrees on

Rule	Where it matters
Distances are metres, time is seconds, speed is metres per second. A kart's top speed is 35 to 40 m/s (about 126 to 144 km/h).	everywhere
The ground is the x / z plane; y is up. Heading `h` 0 points along +z; forward is (sin h, cos h).	`sim.js`, `visuals.js`
Steer +1 is right. A positive sideways value (`lat`) is right of the centre line.	`sim.js`, `ai.js`, `trackBuild.js`
The simulation step is exactly 1/60 s and never depends on the frame rate.	`sim.js`, `Race.jsx`
Stats are on a 1 to 10 scale; `simStats()` is the only place that turns them into physics numbers.	`karts.js`
Colours are painted on the corners of the shapes (vertex colours). No picture files.	all 3D files

4. Building the game step by step

This chapter is the build plan: what to install, how to create the empty project and its folders, and the **order** in which to write the files. The order is the one used for this game, put in a clean sequence. Every phase ends with a check, so you always know the part you just wrote works before you build on it.

What you need

Tool	Why	Where
Node.js 22 or newer (24 was used)	runs Vite, npm and all the test scripts. The game itself only needs Node 18, but the scripts that drive Chrome use the WebSocket built into Node 22.	nodejs.org (LTS)
Google Chrome (or Edge)	to play, and for the browser tests, pictures and this PDF	google.com/chrome
A code editor	for example Visual Studio Code	code.visualstudio.com
Git for Windows (optional)	gives `pdftotext`, used for the page numbers of this PDF	git-scm.com

Creating the empty project

The quickest way to get exactly the same libraries as this game is to copy `package.json` into a new folder and run `npm install`. To start from nothing instead:

```
npm create vite@5 sky-kart-3d -- --template react
cd sky-kart-3d
npm install three@0.169 @react-three/fiber@8 zustand@4 howler@2
npm install -D qrcode-terminal
```

Then make the folders:

```
mkdir src/ui src/game scripts scripts/lib docs docs/guide public/previews
```

Delete the example files Vite made (`src/App.css`, `src/assets/`, the React logo) and replace `vite.config.js` with the one from chapter 2: `base: './'` and the three extra chunks.

The build order at a glance

Phase	You write	Done when
0. Skeleton	`package.json`, `vite.config.js`, `index.html`, `src/main.jsx`, `src/App.jsx`	`npm run dev` shows a page
1. Data	`game/tracks.js` (one track), `game/karts.js`	the files load in Node without errors
2. Track builder	`game/trackBuild.js`, `scripts/check-tracks.mjs`	the check passes and the layout picture looks right
3. Simulation	`game/sim.js`, `game/ai.js`, `game/setup.js`, `scripts/check-sim.mjs`	4 AI karts finish 3 laps in Node

Phase	You write	Done when
4. 3D world	`game/geo.js`, `kartModel.js`, `props.js`, `biomes.js`, `world3d.js`, `visuals.js`, `bus.js`, `input.js`, `src/Race.jsx`	you can drive the track in the browser
5. HUD and touch	`src/Hud.jsx`, `src/Touch.jsx`, `src/styles.css`	position, laps and touch buttons work
6. Rules and save	`game/save.js`, `game/progress.js`, `src/store.js`, `scripts/check-save.mjs`	rewards, stars and unlocking pass their checks
7. Screens	everything in `src/ui/`, then `App.jsx` again	you can go menu → map → race → results → garage
8. Sound	`game/audio.js`	sounds and music play after the first tap
9. All 12 tracks	`game/autoplace.js`, the other 11 tracks, `public/previews/`	all checks pass on 12 tracks
10. Phone and offline	`public/sw.js`, `manifest.webmanifest`, `icon.svg`, `scripts/play.mjs`, `play now.cmd`	the game opens on a phone on the same Wi-Fi
11. Tests, tuning, documents	`check-gameplay.mjs`, `browser-test.mjs`, `bot-balance.mjs`, `perf-report.mjs`, `docs/`	every test passes

Phase 0: the skeleton

- `index.html`: one `<div id="root">` and the script `src/main.jsx`. Add the phone viewport line now (`width=device-width, initial-scale=1, maximum-scale=1, user-scalable=no, viewport-fit=cover`); it is easy to forget later.
- `src/main.jsx`: create the React root and render `<App />`.
- `src/App.jsx`: for now, just a title.

Check: `npm run dev`, open `http://localhost:5173`.

Phase 1: the data

- `game/tracks.js`: start with **one** track: an id, a name, a road half-width (about 9.5 m) and 8 to 12 curve points that make a closed loop a few hundred metres across. Colours can wait.
- `game/karts.js`: the kart list with stats (1 to 10) and `simStats()`, which turns stats into top speed, acceleration and steering speed.

Writing data first means every later file has something real to read.

Check: `node -e "import('./src/game/tracks.js').then(m => console.log(m.TRACK_LIST.length))"` prints 1.

Phase 2: the track builder

`game/trackBuild.js` turns the curve points into a smooth closed curve (a Catmull-Rom spline) and cuts it into **samples every 2 m**. For each sample keep the position, the forward direction, the sideways direction and how curved the road is there. Then add `locate()` (nearest sample and sideways distance for any point), checkpoints and walls.

`scripts/check-tracks.mjs` checks the result: the loop is closed, no part of the road comes too close to another part, corners are not tighter than the road is wide, and the start is straight. With `--layout` it also writes

`scripts/out/layout.html`, a drawing of every track from above. Look at it: it is the fastest way to see a bad track.

Check: `node scripts/check-tracks.mjs --layout` passes.

Phase 3: the simulation

`game/sim.js` is the heart of the game, so it comes before any graphics. Write it in this order, testing as you go:

1. `createWorld()` and a `step(world, input, 1/60)` that only drives forward and steers.
2. Speed limits, braking and reverse, walls that push the kart back, off-road slowdown.
3. Laps with **ordered checkpoints**, then positions.
4. `game/ai.js`: computer drivers that look ahead along the road, slow down for curves and keep their own lane.
5. Drifting (hold drift while steering; the longer you drift the bigger the mini-boost), the boost bar and the boost button, boost pads.
6. Ramps and jumps (height, gravity, landing), surfaces (sand, water, ice, dirt), swinging boulders, kart-to-kart bumps.
7. Stuck and wrong-way recovery, and the end of the race (results list).

`game/setup.js` builds the 4 racers; `scripts/check-sim.mjs` lets the AI drive every kart, the player's too, for 3 laps.

Check: `node scripts/check-sim.mjs` shows 4/4 karts finished, no respawns, sensible lap times.

> **Tip:** Keep `sim.js` free of three.js, React and `window`. Then a whole race runs in Node in a fraction of a second, and every rule can be tested without opening a browser.

Phase 4: the 3D world

1. `game/geo.js`: helpers that make a box, cylinder, cone or sphere **with a colour**, and merge many of them into one shape. One shared material for everything keeps the game fast.
2. `game/kartModel.js`: the kart and driver from about 25 small shapes. Keep the wheels separate so they can spin.
3. `game/world3d.js`: the road as a ribbon along the samples, kerbs, walls, the start gate, then the floating island underneath.
4. `game/props.js` and `game/biomes.js`: scenery models. Place them with **instancing**: one draw call for all palm trees, however many there are.
5. `game/visuals.js`: the moving parts: karts (placed between the last two simulation steps), the chase camera, coins, sky, sun, clouds.
6. `game/input.js` and `game/bus.js`.
7. `src/Race.jsx`: a React Three Fiber `<Canvas>`; inside `useFrame()` read input, run `step()` as many times as needed, call `visuals.update()`.

Check: set `App.jsx` to show `<Race />` directly and drive a lap with the arrow keys.

Phase 5: HUD and touch

- `src/Hud.jsx`: position, lap, timer, coins, a minimap drawn on a small canvas, the boost bar. Read the numbers with a timer 10 times a second, not from React state every frame.
- `src/Touch.jsx`: a joystick that gives analog steering and big buttons. Use pointer events and keep track of each finger, so steering, drift and boost work at the same time.
- `src/styles.css`: start with the HUD and the touch buttons; use `env(safe-area-inset-*)` so nothing hides behind a phone's notch.

Check: in Chrome's device toolbar (phone mode), the buttons react and do not cover the road too much.

Phase 6: rules and saving

1. `game/save.js`: the default save and `sanitize()`. Write `sanitize()` so that it accepts **any** value; the tests will throw garbage at it.
2. `game/progress.js`: `applyRace()` (coins for the place, coins picked up, stars, unlocking the next track, track tasks), the missions and `claimMission()`, and the shop functions. Each one returns a new save.
3. `src/store.js`: the Zustand store with the screen, the save and pause; every action that changes the save writes it to `localStorage` straight away.
4. `scripts/check-save.mjs`.

Check: `node scripts/check-save.mjs`: stars, unlocking, no double reward, purchases and broken saves all pass.

Phase 7: the screens

Write `ui/Shell.jsx` first (logo, coins, bottom bar), then `Menu`, `MapScreen`, `Results` (the core loop: pick a track, race, see the result), then `Garage` with `GaragePreview`, `Missions`, `Pause` with the settings panel, and `SettingsScreen`. Finally make `App.jsx` switch between them by `store.screen`, and load `Race.jsx` lazily so the menus appear quickly.

Check: play the whole loop: menu → map → race → results → next track; buy a kart in the garage; reload the page and see that everything is still there.

Phase 8: sound

`game/audio.js` makes every sound in code: it writes short sound waves (sine, triangle, saw, noise) into WAV data and gives them to Howler. The engine sound changes pitch with speed. Each track gets a short seeded music loop. Browsers only allow sound after the first tap or key press, so `App.jsx` starts audio on the first `pointerdown` or `keydown`.

Check: sound starts after the first click; mute and the volume sliders work.

Phase 9: twelve tracks

With one track working, add the other eleven. Writing pad, ramp and shortcut positions by hand for 12 tracks is slow and easy to get wrong, so `game/autoplace.js` finds them from the road's shape and a short recipe in each track's data. Make each track different: shape, colours, weather, scenery, landmark, surfaces, gravity (Lunar Circuit), music.

Then make the map pictures: `npm run build`, then `node scripts/make-previews.mjs`.

Check: `npm run check` passes on all 12 tracks, and every shortcut is faster than the main road.

Phase 10: phone and offline

- `public/manifest.webmanifest` and `public/icon.svg` for "Add to home screen".
- `public/sw.js`: a service worker that keeps a copy of the game for offline play; register it in `main.jsx` for the built game only.
- `scripts/play.mjs` and `play now.cmd`: serve `dist/` to the local network and print the phone address.

Check: `play now.cmd`, then open the printed address on a phone on the same Wi-Fi.

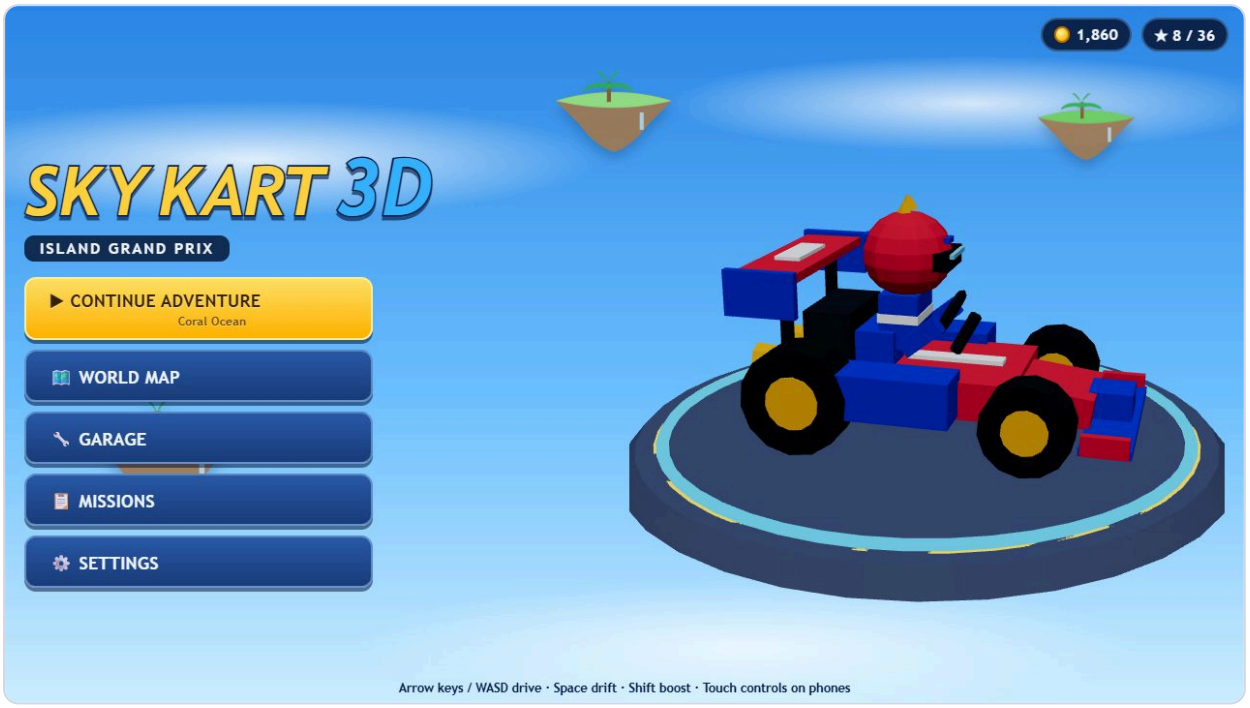
Phase 11: tests, tuning and documents

- `scripts/check-gameplay.mjs` for the driving rules, `scripts/browser-test.mjs` for the real game in Chrome.
- `scripts/bot-balance.mjs` to tune the rivals: early tracks should be easy to win, later ones should need upgrades.
- `scripts/perf-report.mjs` to keep draw calls and triangles low.
- The documents in `docs/`.

Check: `npm run check`, `npm run build`, `npm run test:browser` all pass.

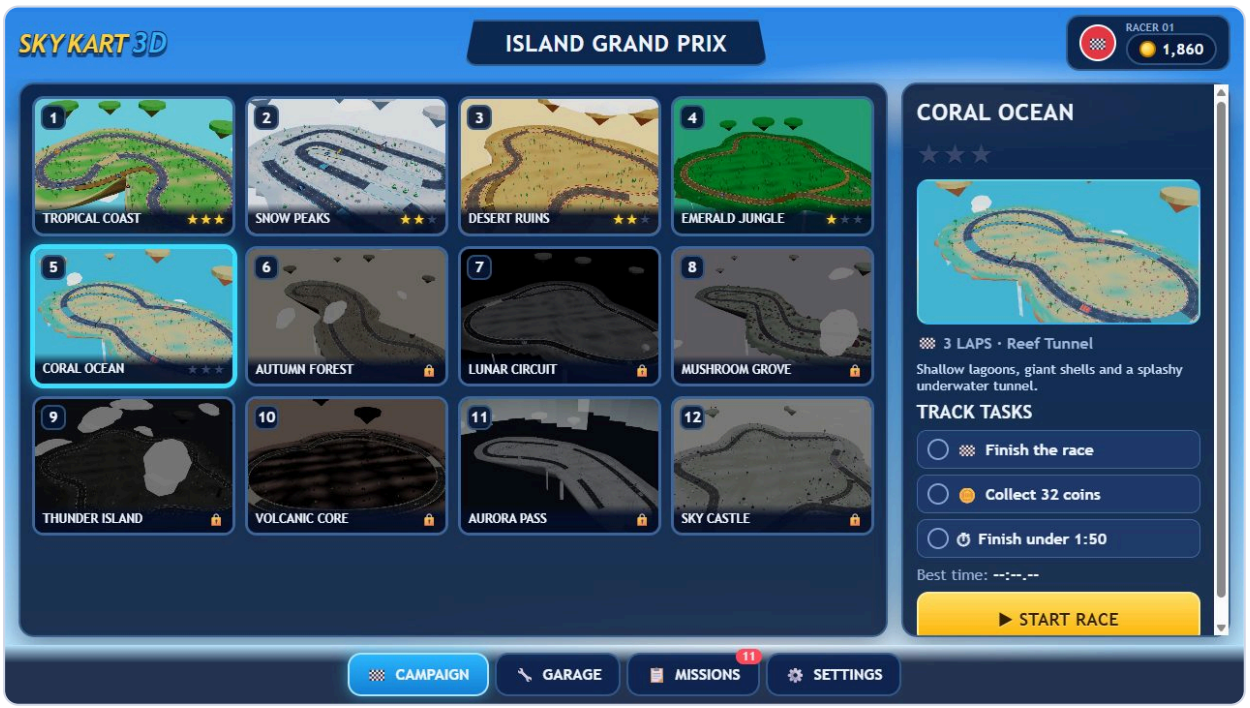
5. What the game can do

The main menu and the map



Main menu: Continue Adventure jumps to the next track you have not finished.

The **world map** shows the 12 tracks as cards with a picture, stars and a lock. Choosing a card shows its details on the side: laps, the shortcut's name, a description, the 3 track tasks, best time and best lap, and START RACE. Only open tracks can be started.



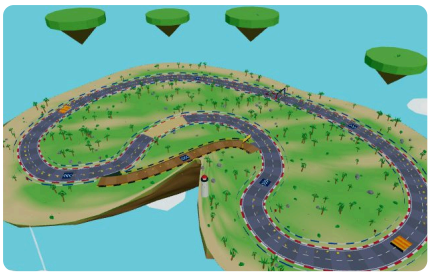
The world map in the middle of the campaign.

The twelve tracks

Every track has its own road shape, colours, weather, scenery, a landmark, music and a shortcut with its own surface and dangers. Lap lengths are 1.2 to 1.7 km.

No.	Track	What makes it different	Weather	Shortcut	Saves
1	Tropical Coast	beaches, palms, a lighthouse	sunny	Sand Spit (dirt)	43 %
2	Snow Peaks	zig-zag mountain road, two hairpins, ice	snow	Ice Slide (ice)	49 %
3	Desert Ruins	long straights, pillars, pyramids, rolling boulders	dust	Temple Cut (dirt)	31 %
4	Emerald Jungle	twisting vines, muddy bends, a temple	fireflies	Vine Bridge (dirt, ramp)	45 %
5	Coral Ocean	lagoons, giant shells, coral	sparkles	Reef Tunnel (water)	37 %
6	Autumn Forest	golden leaves, windmill, fast S-bends	falling leaves	Deer Trail (dirt)	44 %
7	Lunar Circuit	**low gravity**, long jumps, a blue planet in the sky	none	Crater Cut (road, big jump)	19 %
8	Mushroom Grove	giant mushrooms, glowing spores, long hairpin	fireflies	Spore Path (dirt, bounce ramp)	50 %
9	Thunder Island	stormy five-corner circuit, lightning	rain	Bolt Bridge (road)	31 %
10	Volcanic Core	hot rock, lava glow, a volcano	embers	Ash Bridge (dirt, boulder)	17 %
11	Aurora Pass	icy night roads under northern lights	sparkles	Glacier Run (ice)	46 %
12	Sky Castle	the finale: jumps, boulders, a castle	petals	Royal Bridge (road, jump, 2 boulders)	44 %

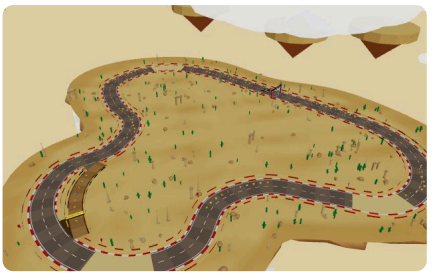
"Saves" is how much shorter the shortcut is than the piece of main road it skips. Over 3 laps the shortcuts save 3 to 18 seconds; they are narrower and harder to drive.



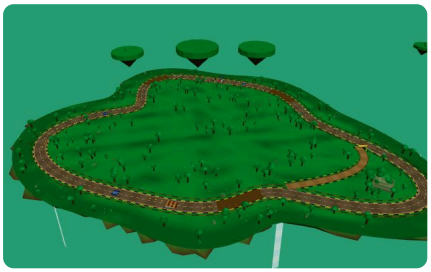
1. Tropical Coast



2. Snow Peaks



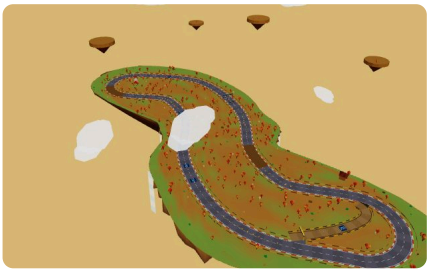
3. Desert Ruins



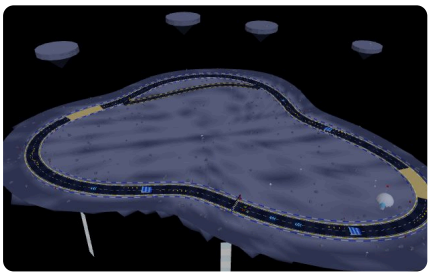
4. Emerald Jungle



5. Coral Ocean



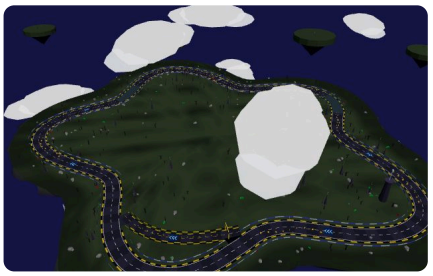
6. Autumn Forest



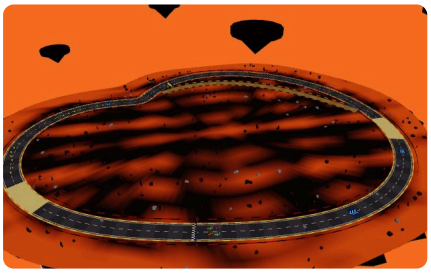
7. Lunar Circuit



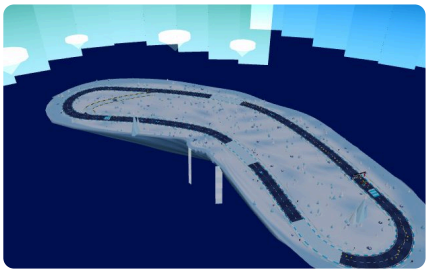
8. Mushroom Grove



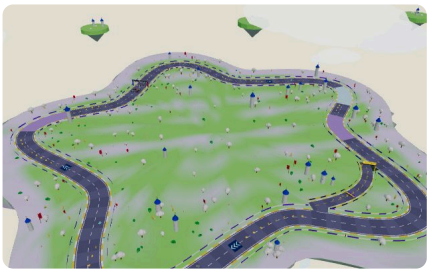
9. Thunder Island



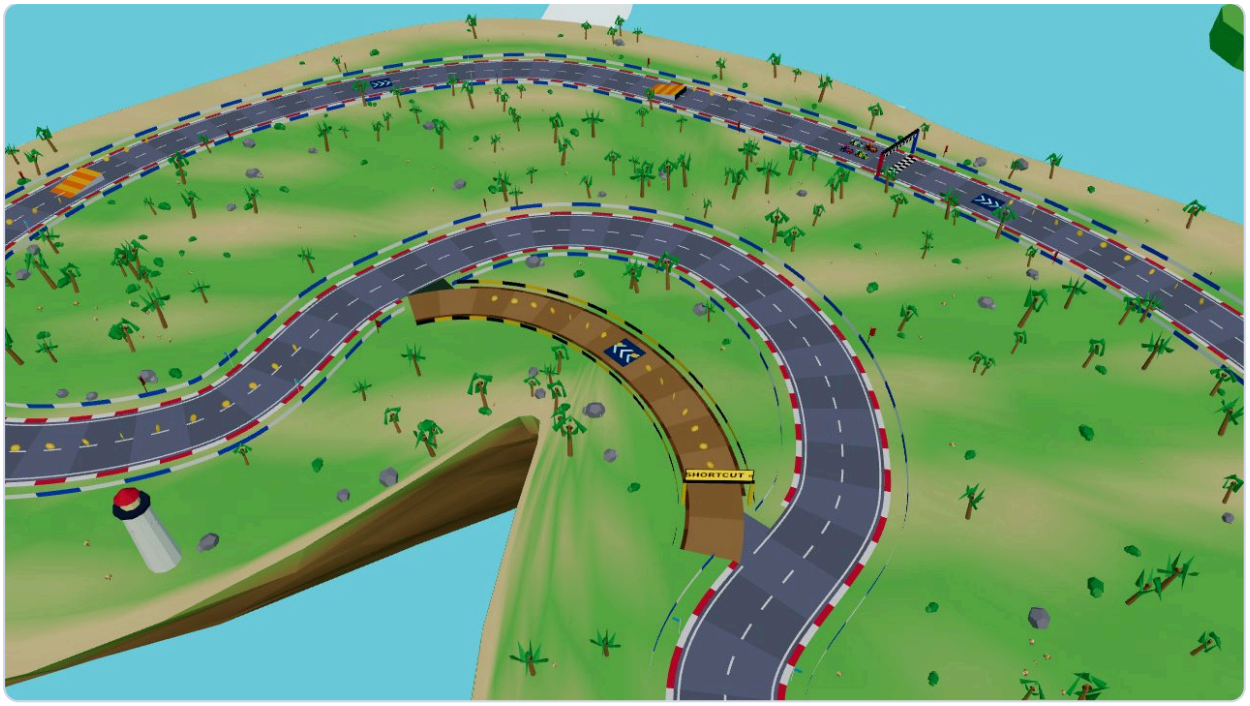
10. Volcanic Core



11. Aurora Pass



12. Sky Castle



Tropical Coast from above: the brown Sand Spit shortcut cuts across the bend.



Lunar Circuit at night, low gravity



Sky Castle, the last track

A race

- 4 karts: you and 3 rivals (Rusty, Mochi, Gus, Vex, Piper or Brick, depending on the track). 3 laps. A 3-2-1-GO countdown.
- **Checkpoints** (blue flags): a lap only counts after all of them, in order.
- **Live position**, lap, timer, coins and a minimap with every kart.
- **Coins** lie on the road and come back every lap. They also fill the boost bar a little.
- **Drifting**: hold drift while steering. Sparks turn blue, then orange, then purple; let go for a mini-boost of 0.75, 1.3 or 2 seconds.
- **Boost bar**: fills while drifting, from coins and pads. Hold BOOST to use it.
- **Boost pads** (blue arrows) and **ramps** that launch the kart into the air.
- **Surfaces**: dirt and water slow you a little, sand slows you a lot, ice is slippery. Grass beside the road is slow.
- **Dangers**: swinging boulders that spin you out, walls, bumping into other karts (heavy karts push light ones).
- **Safe recovery**: a kart that is stuck for 3.5 seconds, or drives the wrong way for 2.8 seconds, is put back on the road. R does it at once.

- **Look back** with C.
- The race ends when everyone has finished, or 8 seconds after you did.

Stars, rewards and unlocking

You finish	Stars	Coins for the place
1st	3	150
2nd or 3rd	2	100 or 60
4th	1	30
did not finish	0	0

- **Finishing a track in any place unlocks the next one.**
- Stars never go down. Each **new** star pays 100 coins, once.
- Each track has 3 **tasks**: finish the race, collect N coins (25 to 40) in one race, finish under a time (1:45 to 2:30). Each task pays 50 coins the first time.
- Coins picked up during the race are added too.
- Best time and best lap are kept per track.



Results: position, stars, the finishers, every reward line, the tasks and the next steps.

Missions

20 missions count everything you do in all races. When one is complete, press **CLAIM** to get its coins. Each can be claimed only once.

Mission	Reward	Mission	Reward
Collect 20 coins	100	Jump off 5 ramps	150
Collect 150 coins	250	Take 5 shortcuts	250

Mission	Reward	Mission	Reward
Collect 600 coins	600	Finish a race	100
Drift for 10 seconds	100	Finish 10 races	400
Drift for 60 seconds	250	Win a race	200
Drift for 5 minutes	600	Win 5 races	500
Use 3 boost pads	150	Win 12 races	1 200
Use 30 boost pads	350	Finish a race in under 2:30	300
Boost for 20 seconds	200	Finish a race in under 2:00	600
Earn 12 stars	400	Earn 30 stars	1 000



The missions screen. The red number on the Missions button counts rewards waiting to be claimed.

The garage

Kart	Price	Speed	Accel	Handling	Boost	Weight	In short
Comet	free	5	5	5	5	3	balanced, the first kart
Sprout	700	4	7	8	5	2	light, loves tight corners
Bulldog	1 400	8	4	4	5	5	heavy, huge top speed, pushes rivals
Stardust	2 400	6	6	6	9	3	fills the boost bar very fast
Skyhawk	4 200	8	7	7	7	3	the champion, strong at everything

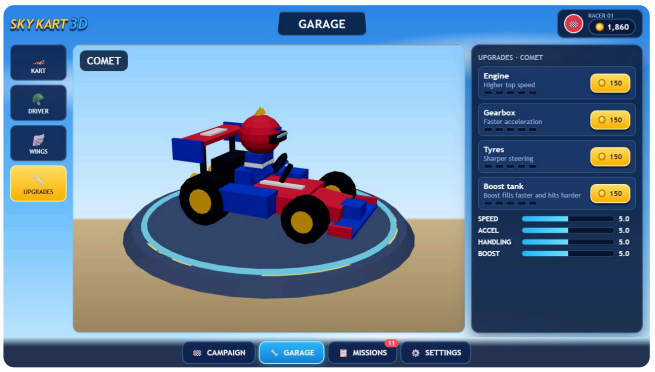
Upgrades (per kart, 5 levels each, +0.6 stat points per level): Engine (top speed), Gearbox (acceleration), Tyres (steering), Boost tank (boost fills faster and pushes harder). Levels cost 150, 250, 400, 600 and 900 coins. The stats really change the driving: top speed goes from 35 to 40 m/s across the scale, and the tests show better stats give faster laps.

Looks (no effect on driving). Each kart keeps its own paint.

Kind	Free	Costs coins
8 paints	Crimson, Solar, Lime	Ocean 150, Violet 150, Sunset 250, Gold 400, Midnight 400
5 drivers (helmet colours)	Ace	Nova 120, Kai 120, Rio 200, Jet 300
4 wings	Classic	Sleek (no wing) 100, Twin 180, Fin 250



Kart tab: stats, karts, paint



Upgrades tab

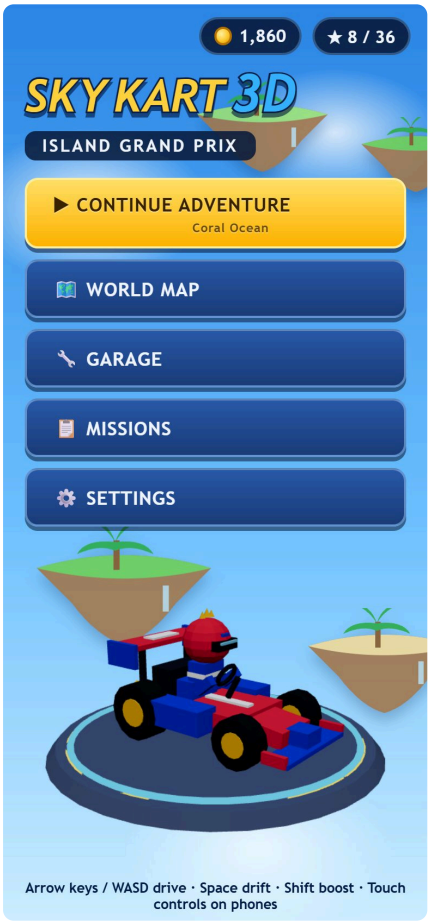
Controls

Action	Keyboard	Gamepad	Touch
Accelerate	Up arrow / W	RT or A	automatic (or GAS button)
Brake / reverse	Down arrow / S	LT or B	BRAKE
Steer	Left / Right, A / D	left stick	joystick (analog)
Drift	Space	RB	DRIFT
Boost	Shift or B	X	BOOST
Look back	C	LB	
Put back on road	R		
Pause	Esc or P	Start	pause button

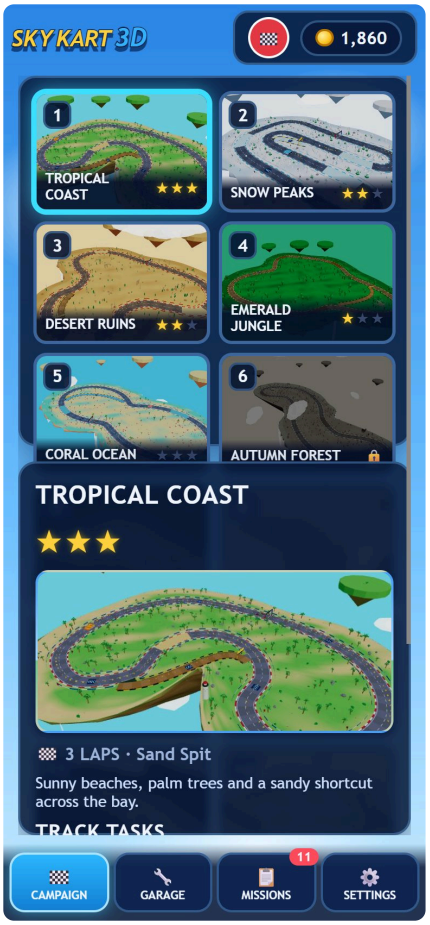
Touch controls appear on phones and tablets (and in any window narrower than 720 pixels). Several fingers work at once. **Auto-accelerate** is on by default on touch, so you only steer, drift, boost and brake; turn it off in Settings to get a GAS button.



A phone held sideways: joystick on the left, DRIFT, BOOST and BRAKE on the right.



Phone, portrait: menu



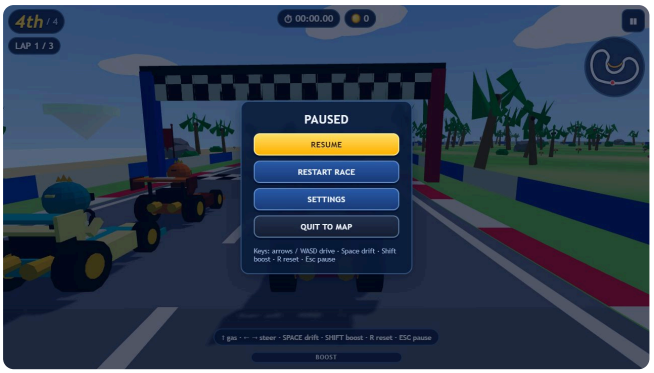
Phone, portrait: map



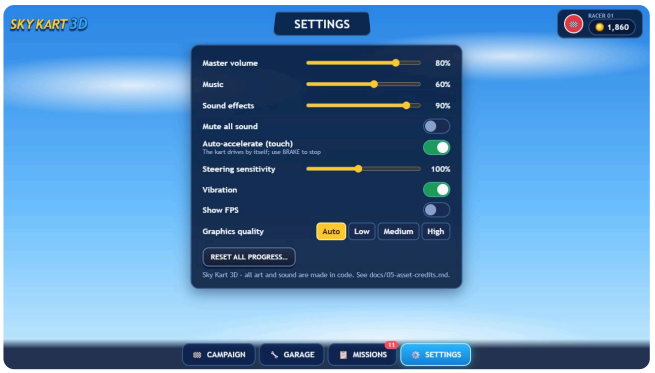
Phone, portrait: garage

Pause and settings

The race pauses with Esc, P or the pause button, and **by itself** when the window loses focus or the tab is hidden. Every held key and finger is released, and the race clock stops.



Pause: Resume, Restart race, Settings, Quit to map



Settings

Settings: master, music and effects volume; mute; auto-accelerate; steering sensitivity; vibration; show FPS; graphics quality (Auto, Low, Medium, High); reset all progress.

Graphics quality "Auto" picks Low on phones with 4 or fewer processor cores or 3 GB or less memory, Medium on other phones and on PCs with fewer than 8 cores, and High otherwise.

Quality	Picture sharpness limit	Shadows	Smooth edges	Scenery	Sparks / weather
Low	1 ×	no	no	45 %, fewer far islands, fog a little closer	160 / 90
Medium	1.5 ×	yes (1024 px)	no	75 %	320 / 220
High	2 ×	yes (2048 px)	yes	100 %	520 / 220

Sound and music

Every sound is made by the game's code when it starts: engine (its pitch follows the speed), drift, coins, boost, mini-boost, jumps, landings, bumps, hits, countdown, lap, finish and menu clicks. Each track has its own short music loop, and the menus have their own tune.

Saving

Coins, stars, best times, unlocked tracks, task and mission progress, claimed rewards, owned karts, upgrades, paints, drivers, wings, the kart in use and all settings are saved in the browser after every change. Nothing is sent to any server.

6. Running, building and hosting

Play on your Windows PC

1. Install **Node.js** (LTS) from nodejs.org, once.
2. Double-click `play now.cmd` in the project folder.

What it does, in order:

Step	What happens	Time
1	Checks that Node.js 18 or newer is installed and explains what to do if not.	instant
2	The first time only: `npm install` downloads the libraries (needs internet).	1 to 2 minutes
3	`npm run build` makes a fresh `dist/` (errors are shown if the build fails).	a few seconds
4	`node scripts/play.mjs` serves the game, opens your browser and prints the phone address and a QR code.	instant

Keep the window open while you play. Close it, or press Ctrl+C, to stop. `play now.cmd --no-open` starts without opening a browser tab.

Play on your phone

1. The phone must be on the **same Wi-Fi** as the PC (not a guest network).
2. Start `play now.cmd` on the PC.
3. Scan the QR code with the phone's camera, or type the "On your phone" address (for example `http://192.168.1.20:4173/`).
4. The first time, Windows may ask whether Node.js may use the network: tick **Private networks** and click **Allow**.
5. Turn the phone sideways for the best view. In the browser menu, "Add to Home screen" gives it its own icon and full screen.

> **Note:** If the first address does not open, the window lists other addresses of the PC to try. A VPN on the PC or the phone can block the connection.

All the commands

Run them in a terminal in the project folder.

Command	What it does
`npm install`	Downloads the libraries into `node_modules/`. Needed once, and again after `package.json` changes.
`npm run dev`	Development server at `http://localhost:5173` that reloads as you change the code. Add `-- --host` to open it from a phone too.
`npm run build`	Makes the finished game in `dist/`.
`npm run preview`	Serves `dist/` on this PC only, to check the build.

Command	What it does
`npm run play`	Serves `dist/` on this PC and the Wi-Fi (what `play now.cmd` uses).
`npm run check`	The four Node test programs (tracks, save and rules, AI races, gameplay). About 2 minutes.
`npm run test:browser`	The full browser test in Chrome (needs a fresh build). About 10 minutes. `node scripts/browser-test.mjs --quick` skips the 12-track loop.
`npm run balance`	The difficulty check with simulated players. `KART=skyhawk UPG=2` in front tests another kart and upgrade level.
`npm run previews`	Remakes the map pictures in `public/previews` (needs a fresh build, then build again to copy them into `dist/`).
`npm run guide`	Makes this PDF from `docs/guide/`. `node scripts/guide-shots.mjs` remakes its pictures first.

If Chrome is not in its usual folder, set `CHROME_PATH` to `chrome.exe` before the browser scripts.

Putting the game online

`npm run build` makes `dist/`, a plain static website: HTML, JavaScript, CSS and pictures. **Upload the contents of `dist/`** to any static host. No server program, database or account is needed.

Host	How
Netlify	Drag the `dist` folder onto app.netlify.com/drop.
Vercel	Import the project; framework "Vite", build command `npm run build`, output folder `dist`.
GitHub Pages	Put the contents of `dist/` on a branch of their own (for example `gh-pages`) and turn on Pages for that branch.
Cloudflare Pages	Build command `npm run build`, output folder `dist`.
Your own server	Copy the files in `dist/` into any folder served by nginx, Apache or IIS.

Things worth knowing:

- The game uses **relative paths** (`base: './'`), so it works at the root of a site **or in a sub-folder** like `example.com/games/skykart/`.
- Use **https** if you can: offline play (the service worker) and keeping the screen awake during races only work on https (or on localhost).
- Files in `dist/assets/` have a code in their name and can be cached for a long time. `index.html` and `sw.js` should not be cached for long, so players get new versions.
- Nothing is sent anywhere. There are no accounts, no analytics and no ads. Each player's progress stays in their own browser.

> **Note:** The game has **not** been put online. Nothing was uploaded or published.

When something goes wrong

Problem	What to do
"Node.js was not found"	Install the LTS version from nodejs.org, then run `play now.cmd` again.

Problem	What to do
"Could not install the game's libraries"	Check the internet connection, then run it again.
The phone cannot open the address	Same Wi-Fi? Allow Node.js through the Windows firewall (Private networks). Try the other addresses printed. Switch off VPNs.
The game is slow or hot on a phone	Settings → Graphics quality → Low. Close other apps. Turn on "Show FPS" to see the frame rate.
No sound	Tap the screen once (browsers block sound before a tap). Check Mute and the volume sliders, and the phone's silent switch.
Progress disappeared	Progress is stored per browser and per address. A different browser, a private window, clearing site data, or a different address (localhost vs. the Wi-Fi address) each have their own save.
Browser scripts fail with "Chrome did not start"	Install Chrome or set `CHROME_PATH`. Make sure the disk is not full.

7. Testing

Every part of the game is checked by a program, so a change that breaks something shows up at once. The tests were run again on 23 October 2026 for this guide, on Windows 11 with Node 24 and Google Chrome.

The test programs and their results

Program	Command	What it checks	Result
Track geometry	`node scripts/check-tracks.mjs`	the shape of all 12 tracks	all 12 pass
Save and rules	`node scripts/check-save.mjs`	saving, repair, rewards, missions, shop	27 of 27 pass
AI races	`node scripts/check-sim.mjs`	a full race on every track	12 of 12 tracks: all 4 karts finish
Gameplay	`node scripts/check-gameplay.mjs`	laps, driving, stats, shortcuts, speed	31 of 31 pass
Browser	`node scripts/browser-test.mjs`	the real game in Chrome, PC and phone	76 of 76 pass
Build	`npm run build`	the finished game	builds with no warnings

The first four run in Node.js without a browser (`npm run check` , about 2 minutes). The browser test needs a fresh build and Chrome (about 10 minutes).

What each program checks

Track geometry (`check-tracks.mjs`), for every track: the loop is closed and smooth; no part of the road comes closer than two road-widths to another part; corners are not tighter than the road allows; the start is straight; checkpoints are in order and none is on the shortcut; the shortcut is shorter than the road it skips and keeps clear of the rest; there are at least 2 boost pads and 1 ramp; every coin lies on a road; the tasks are set.

Track	Lap (m)	Tightest corner (m)	Checkpoints	Pads	Ramps	Moving obstacles	Shortcut vs. road it skips (m)
Tropical Coast	1 327	29	6	4	2	0	139 instead of 244
Snow Peaks	1 248	24	6	3	2	2	75 instead of 148
Desert Ruins	1 169	29	6	4	1	4	94 instead of 136
Emerald Jungle	1 560	21	6	3	2	3	179 instead of 328
Coral Ocean	1 222	16	5	2	2	3	238 instead of 376
Autumn Forest	1 662	33	6	3	2	3	143 instead of 256
Lunar Circuit	1 651	67	6	2	3	3	305 instead of 376
Mushroom Grove	1 636	36	6	3	2	2	139 instead of 280
Thunder Island	1 577	33	6	4	2	4	135 instead of 196
Volcanic Core	1 258	21	6	4	2	5	310 instead of 376

Track	Lap (m)	Tightest corner (m)	Checkpoints	Pads	Ramps	Moving obstacles	Shortcut vs. road it skips (m)
Aurora Pass	1 666	55	6	4	2	3	178 instead of 328
Sky Castle	1 390	27	6	4	3	4	103 instead of 184

Save and rules (`check-save.mjs`): garbage, broken and hostile saves (wrong types, negative or huge coins, unknown tracks and karts, too many stars, upgrade level 99) all become valid saves; broken text is kept aside; a blocked browser storage never crashes the game; stars are 1 / 2 / 3; finishing in last place still unlocks the next track; not finishing gives nothing; star and task coins are paid only once; best times only get better; missions cannot be claimed early or twice; purchases charge once, refuse when you are short of coins or already own the item, and stop at level 5; only owned items can be equipped; the real store writes to storage at once.

AI races (`check-sim.mjs`): on every track all four karts, driven by the AI, finish 3 laps with valid lap times and no karts lost off the track. Winning times are between about 90 and 140 seconds.

Gameplay (`check-gameplay.mjs`):

- A lap is **not** counted when a kart jumps to the end of the lap without the checkpoints.
- Checkpoints are reported in order; finishing positions are 1, 2, 3, 4 in order of time; the live order always matches progress.
- Ramps launch the kart and it lands; jumps on Lunar Circuit last longer (low gravity).
- The boost button only works with boost in the bar, empties it, and makes the kart faster.
- Drifting fills the bar and gives a mini-boost when released; boost pads are counted for the missions.
- Walls keep karts on the island; braking stops and then reverses; a stuck kart is put back, and so is one after R.
- Karts bump each other apart.
- **Better stats give faster laps** (speed, acceleration, handling and boost each tested).
- On **every one of the 12 tracks the shortcut is used, can be finished, and is faster** than the main road (3 to 18 seconds over 3 laps).
- One simulation step with 4 karts takes about 0.003 ms.

Browser (`browser-test.mjs`), in a real Chrome against the built game:

- Menus: every main button, 12 track cards with 11 locked at the start, a locked track cannot be started.
- Real key presses: gas, steering, drift and boost together, brake.
- Pause with Esc; the race clock stops; held keys are released; losing focus pauses; settings from the pause menu.
- A whole race to the end: coins are added and saved, the next track opens, the best time is stored; everything is still there after reloading the page; Continue Adventure picks the right track.
- Missions: CLAIM pays once, and stays claimed after a reload.
- Results and pause buttons: NEXT TRACK, REPLAY, MAP, Restart race, Quit to map, the logo back to the menu.
- Garage: buying a kart, an upgrade and a paint, equipping, all still there after a reload; **the bought upgrade really changes the kart's top speed in the race**.
- A broken save and a nonsense save do not crash the game; settings are saved.
- **All 12 tracks** are started, driven to the end by the autopilot and show results.
- No errors or warnings in the browser console.

- Phone emulation (844 × 390 and 390 × 844): touch buttons are big enough; three fingers at once (steer + drift + boost); the joystick is analog; brake beats auto-gas; letting go clears everything; losing focus while a button is held releases it; turning the phone resizes the game; no screen sticks out sideways in portrait.

Difficulty

`bot-balance.mjs` races simulated players on every track. With the free Comet kart and no upgrades, a "good" player that uses the shortcut and boost wins tracks 1 to 3 every time, fights for places on tracks 4 to 8 and finishes last on 9 to 12. With Skyhawk and 2 upgrade levels it wins almost everywhere. So the later tracks are meant to need upgrades or a better kart.

> **Honest status:** These are simulated players, not people. Real players who drift well may find the game easier; new players may find it harder.

Speed

`perf-report.mjs` drives every track at each quality level in a phone-sized window and reads what the graphics card has to draw.

Quality	Most draw calls	Most triangles	Most JavaScript time per frame
Low	110	77 000	0.28 ms
Medium	106	103 000	0.18 ms
High	106	120 000	0.25 ms

Draw calls and triangles are the same on every device. The JavaScript time was measured on this PC. A phone is slower; even if it needed ten times longer, that would still be small next to the 16.7 ms a frame may take at 60 FPS.

> **Not tested yet:** The **frame rate** was not measured on any real device. The test Chrome draws without a graphics card (software drawing), so its FPS says nothing about a phone or a gaming PC. The game protects itself (shared colour-only material, instanced scenery, quality levels, automatic resolution drop below 48 FPS), but only a real phone can show how smooth it is.

What has NOT been tested

Not tested	Why it matters
A real phone or tablet	Touch was only emulated. Real fingers, frame rate, heat and battery are unknown.
Safari (iPhone, iPad, Mac), Firefox, Edge	Only Chrome was used.
Listening to the sound	All sounds play without errors, but nobody has judged how they sound.
A real gamepad	The code is there; no gamepad was connected.
Offline play	The service worker installs, but starting the game with no internet was not tried.
The game online	It was never uploaded to a host.
Real players	Difficulty was tuned with simulated players only.

How to run all the tests again

```
npm install
npm run check
npm run build
npm run test:browser
node scripts/perf-report.mjs
npm run balance
```

8. Changing and extending the game

Most changes touch only one or two files. This chapter says which ones, and which check to run afterwards.

Where to change what

I want to change...	File	Then run
A track's shape, colours, weather, scenery, music, tasks	`src/game/tracks.js`	`node scripts/check-tracks.mjs --layout`, `npm run check`
How many pads, ramps, hazards a track gets	the `auto` recipe of that track in `tracks.js`	`node scripts/check-tracks.mjs`
Kart prices, stats, weights; paints, drivers, wings	`src/game/karts.js`	`node scripts/check-save.mjs`
How stats turn into speed, acceleration, steering, boost	`simStats()` in `src/game/karts.js`	`node scripts/check-gameplay.mjs`
Upgrade prices or the number of levels	`UPGRADE_COST`, `MAX_UPGRADE` in `karts.js`	`node scripts/check-save.mjs`
How strong the rivals are on each track	`rivalStats()`, `rivalDifficulty()` in `karts.js`	`npm run balance`
Coins for each place, stars, task coins	`PLACE_COINS`, `STAR_COINS`, `TASK_COINS` in `src/game/progress.js`	`node scripts/check-save.mjs`
Missions	`MISSIONS` in `progress.js`	`node scripts/check-save.mjs`
Drift, boost, gravity, pad strength	constants at the top of `src/game/sim.js`	`node scripts/check-gameplay.mjs`
How the AI drives	`src/game/ai.js`	`npm run check`, `npm run balance`
Keys and gamepad buttons	`src/game/input.js`	try it in the browser
Touch button sizes and places	`src/styles.css` (`.tbtn`, `.btns`, `.joy`)	`node scripts/screens.mjs` at phone sizes
Colours and look of the menus	`src/styles.css`	`node scripts/screens.mjs`
Sounds and music	`src/game/audio.js`	listen in the browser
What a quality level does	`src/game/visuals.js`, `src/game/world3d.js`, `src/store.js` (`detectTier`)	`node scripts/perf-report.mjs`

After any change: `npm run check`, `npm run build`, and if a screen changed, `npm run test:browser`.

How to add a new track

1. Copy one track block in `src/game/tracks.js` and give it a new `id` and `name`.
2. Change `pts`, the curve points (in metres, going around the loop). The start line is at the first point, and the road must be straight from about 60 m before it to 50 m after it (the check tells you if it is not). Use `scale` to make the whole track bigger or smaller.
3. Set the `theme` (colours and `weather`), `props` (scenery models from `props.js`), `landmarks`, `music` and `tasks`.

4. Set the recipe in `auto`: number of boost pads and ramps, zones (sand 1, water / mud 2, ice 3), moving obstacles, and `scT`, roughly where on the lap (0 to 1) the shortcut should be. Give the shortcut a `name`, `note` and `surface`.
5. Run `node scripts/check-tracks.mjs --layout` and open `scripts/out/layout.html`. Fix any reported problem (usually: points too close together, or a corner too tight).
6. Run `npm run check`: the new track must also pass the AI race and the shortcut test.
7. `npm run build`, then `npm run previews` and `npm run build` again for its map picture.

The map, the save, the stars and the missions pick up the new track by themselves. The number of tracks is `TRACK_LIST.length` everywhere.

How to add a new kart

1. Add an entry to `KARTS` in `src/game/karts.js`: `id`, `name`, `price`, `weight` (1 to 5), `base` stats (1 to 10), a default `paint` and a short `blurb`.
2. Run `node scripts/check-save.mjs`: the "every kart can be bought" check covers it.

The garage shows it, the save accepts it and the race uses its stats with no other change.

How to add a new mission

Add an entry to `MISSIONS` in `src/game/progress.js`: an `id`, the `text`, the `stat` it counts (`coins`, `driftSec`, `pads`, `boostSec`, `ramps`, `shortcuts`, `races`, `wins`, `fastest` or `stars`), a `target` and a `reward`. To count something new, add it to `totals` in `save.js` and fill it in `applyRace()`.

How to add a new screen

1. Make `src/ui/MyScreen.jsx` using `Shell` for the frame.
2. In `src/App.jsx`, show it when `screen === 'myscreen'`.
3. Open it from a button with `useStore.getState().go('myscreen')`, or add it to the bottom bar in `Shell.jsx`.

Known limits

- The ground is flat; only ramps have height. Islands, waterfalls and clouds are scenery.
- Walls are solid and gentle: you cannot fall off an island.
- Paint, drivers and wings only change the look.
- Missions count totals over all races; they do not reset daily.
- The AI cannot drift, does not collect coins, and does not aim for ramps.
- There are no items (shells, shields), no time trial, no multiplayer.
- In a private window or with storage blocked, the game works but progress is lost when the page closes (a warning is shown).

Ideas for later

- Time trial with a ghost kart; more racers; split screen on one PC.
- Items on the road (shells, shields, bananas).

- Medals per track and online leaderboards (would need a server).
- Hand-made 3D models and textures instead of code-made ones.
- Testing on real phones (first priority), then tuning the touch buttons and the difficulty with real players.

9. Words used in this guide

Word	Meaning
Analog steering	Steering that can be a little or a lot (any value from -1 to 1), not only full left or full right.
Asset	A file the game uses that is not code: a picture, sound, model or font. This game has almost none; it makes them in code.
Build	Turning the source code into the finished game (`npm run build` → `dist/`).
Checkpoint	A point on the road every kart must pass, in order, for the lap to count.
Chunk	One of the JavaScript files in `dist/assets/` . Splitting the code into chunks lets the menus load before the 3D engine.
Component	A piece of the screen written in React, for example `Menu.jsx` .
Draw call	One request to the graphics card to draw something. Fewer draw calls means a faster game, especially on phones.
Drift	Sliding sideways through a corner on purpose. Holding a drift long enough gives a mini-boost.
FPS	Frames per second: how many pictures the game draws each second. 60 is smooth.
Headless Chrome	Chrome running without a window, driven by a program. Used for the tests and the pictures.
HUD	Heads-up display: the numbers drawn on top of the race (position, lap, time, coins, minimap, boost bar).
Instancing	Drawing many copies of the same model (for example 300 palm trees) in one draw call.
`localStorage`	A small storage space in the browser for each website. The save lives there.
Low-poly	3D models made from few, flat faces. Cheap to draw and a clear, toy-like look.
Node.js	A program that runs JavaScript outside the browser. It runs Vite, npm and the tests.
npm	The tool that comes with Node.js to download libraries and run the project's commands.
Progress score	A number that says how far a kart is in the race (laps and distance). Positions are sorted by it.
React	A library for building screens from components.
React Three Fiber (R3F)	A library that lets React manage a three.js 3D canvas and runs the frame loop.
Rubber band	A trick that keeps races close: rivals far ahead of you drive a little slower and rivals far behind a little faster (at most 7 %).
Sample	One point on the road's centre line. The road has a sample every 2 metres.
Seeded	Random, but the same every time for the same starting number (seed). Tracks are placed with a seed, so they never change by themselves.
Service worker	A small script the browser keeps for a website. Here it lets the game start without internet.
Shortcut	A second, shorter, narrower road on every track.
Simulation	The part of the game that works out where every kart is and what happens. It does not draw anything.
Spline	A smooth curve through a list of points. Each track's road is one closed spline.
Static host	A web host that only serves files, with no program running on the server. Enough for this game.
Step	One tick of the simulation, exactly 1/60 of a second.

Word	Meaning
three.js	The 3D library that draws the game with WebGL.
Vertex colours	Colours stored on the corners of a 3D shape instead of in a picture.
Vite	The development server and build tool.
WebGL	The part of the browser that lets a web page use the graphics card.
Zustand	A very small library for app state, used in `store.js`.